

Operationalising Machine Learning for Student Performance Prediction: A Cloud-Native MLOps Implementation

Tiana Collingwood
School of Computer Science and Information Technology
Luzern University of Applied Sciences and Art
Rotkreuz, Switzerland
tiana.collingwood@stud.hslu.ch

Link to GitHub Repository: https://github.com/tccmc8/ARTIFIN_project.git

Abstract—With the increased usage of AI and vibe coding, an understanding of the Machine Learning Operations (MLOps) pipeline has become increasingly useful. This paper explores an end-to-end MLOps pipeline for a machine learning model through modelling, deployment and monitoring, CI / CD and cloud deployment. In this report this process was done on a Kaggle Dataset that provides student habits, their lifestyle and their grades. Different Machine Learning algorithms were tested, like Ridge Regression and Random Forest, tracked by MLflow using metrics like Mean Absolute Error, Root Mean Squared Error, and the Coefficient of Determination (r^2) to find the best model. A basic UI on FastAPI displays the prediction versus the actual score. After model is deployed, distribution drifts are monitored and visualised using Grafana. The model is then deployed on Google Cloud Platform (GCP) with a user facing UI that is accessible and updates alongside model updates. For the model to be reproducible, the pipeline is containerised using Docker and a Docker image is saved on GitHub that can be downloaded and used on other devices. This automation is monitored on GitHub and tests any pushes to the repository before adding it to production.

Keywords—MLOps, Machine Learning, Prediction, Docker, Google Cloud, Monitoring, Reproducibility

I. INTRODUCTION

Over the past decade, machine learning has transitioned from a primarily academic research field into a core capability for real-world business and operational systems across many industries [1]. This transition has exposed a persistent gap in ML which is, “ML in production is very different from ML in research” [2]. The trained model itself is a small part of a production ML system, there are multiple other components like data pipelines, serving infrastructure, monitoring, and continuous retraining [3]. Operational challenges occur at every stage of this lifecycle, not merely at the modelling step [1]. The discipline of Machine Learning Operations (MLOps) has emerged to systematically address these challenges.

MLOps unifies “ML system development (Dev) and ML system operation (Ops)” [4], it is a practice that implements automation and monitoring at each stage of the process (integration, testing, deployment, and infrastructure management). It solves real-world issues across industries [1] and this is through the deployment, infrastructure, and monitoring pipeline; the code is only a small fraction of the process [3]. While accuracy and testing remains essential, MLOps must additionally address new challenges, absent from traditional software systems, such as model degradation and data drift, which can be rectified through continuous

learning [2]. This results in multiple deployments, as new data and input is used to update the model, an iterative process which faces its own challenges.

In this report, the student exam results prediction model is used as a vehicle to demonstrate the research to production pipeline, how models are operationalised and handle continuous updates during production highlighting the challenges that occur when running ML systems in real environments as well as demonstrating how MLOps practices integrate into a working end-to-end pipeline. The purpose of this report is demonstrative, and the remainder of the report’s structure mirrors the MLOps lifecycle of data, modelling, deployment and monitoring [5]; methodology which includes modelling, system architecture and deployment; experimental results; discussion and limitations; and conclusion.

A. Related Work

1) MLOps

ML is being increasingly used in production which has led to the popularity of MLOps which extends DevOps principles [6] to ML processes: data ingestion and transformation, continuous integration and deployment, and output to the end-user [7] with the goal of aiding the deployment and management of the model [8] which in turn improves the efficiency and accuracy of the process [9]. Mäkinen et al. (2021) describe MLOps as a process that pushes for automation and monitoring throughout an ML development and system, including monitoring, testing, release management, and infrastructure operations [10]. The focus on deployment and maintenance helps improve the sustainability of ML systems creates a scalable approach that can be used in production environments [11]. Studies emphasise that MLOps bridges the gap between research and production, Kumara et al. notes that MLOps aims to support the continuous delivery of high-quality ML models [12], while Faubel et al. on the other had states that only a small proportion of ML projects achieve continuous production use [8]. This suggest that operational challenges remain a significant barrier to realizing business value from machine learning initiatives. Furthermore, Panchal et al. (2024) argue that operationalizing AI systems requires expertise spanning data engineering, software development, cloud computing, and DevOps practices, illustrating the multidisciplinary nature of modern ML deployment [13] but Poloskei narrows this down to model development, evaluation, deployment and monitoring [14]. MLOps acts as a bridge between research and production using DevOps principles to create a flexible

and reliable process that can handle repetitive testing, deployment and monitoring at each step of the process.

2) CI / CD Pipeline

Continuous Integration and Continuous Deployment (CI/CD) is a foundational component of MLOps that automates the ML pipeline [15]. Repetitive tasks are automated [6], which plays a pivotal role in accelerating model development, deployment, scaling, monitoring, and retraining [16]. As well as managing datasets, model artifacts, and training workflows. A model may need to be retrained and CI / CD allows for models to be retrained automatically when new data is added, which in turn allows for rapid and reliable updates of the pipeline in production [16], a scalable system. Research consistently identifies automation as a critical requirement for scalable ML operations. Studies report that automated pipelines improve reliability, reduce manual intervention, and enable rapid updates to production systems [17]. Sundaram highlights that CI/CD automation allows models to be refreshed as populations and operating conditions evolve [18], while Pölöskei argues that CI/CD pipelines facilitate rapid prototyping and deployment of data-driven applications [14]. Garg et al. further note that advanced MLOps maturity levels rely on fully automated CI/CD pipelines to ensure rapid and reliable updates in production environments [16]. The automation process is done with the goal of reducing human error and intervention, maintaining consistency throughout the pipeline [19]. This testing, versioning and deployment process is done in this project in GitHub actions, a useful tool for CI / CD [17]. Each phase in the MLOps pipeline is run through the CI pipeline [19], having to pass a series of tests before it is deployed and turned into a reusable docker image.

3) Docker and Containerisation

Containerisation has become a standard approach for packaging and deploying machine learning applications to ensure reusability, composability, and portability across environments [16]. ML systems often consist of multiple interconnected components, including preprocessing modules, training environments, inference services, and monitoring tools, a containerisation approach packages the application, along with its dependencies, so it is transportable [14]. In this report, Docker is used as the containerisation technology for this MLOps ecosystem, a commonly adopted technology across ML environments [11], [19] to create container images which allows for the packages application to run on small isolated environments [16] which helps to maintain consistency between development, testing, and production systems. As the report's model is going to be deployed on Google Cloud, it needs to be cloud native hence the importance of Docker as "containers and micro services are the fundamental characteristics of cloud-native applications" [14] as microservices allow for elements to be updated in their own container without impacting the rest of the application and containers are lightweight and streamline making them resource efficient and easily scalable. Docker and containerisation is a state-of-the-art virtualization platform that creates portable bundles containing applications and their dependencies [14]. This portability improves reproducibility, simplifies deployment, and reduces environment-related failures. Docker has become a core technology for modern MLOps implementations and cloud-native ML architectures.

4) Cloud Deployment

As seen in the Docker and Containerisation section, machine learning systems scale, through managing large numbers of containers becomes increasingly challenging. Garg et al. observe that large-scale ML applications may require thousands of containers, making orchestration essential for effective deployment and management. Kubernetes has emerged as the dominant orchestration platform for addressing these challenges [16]. Deploying and operationalising ML systems at this scale is not straightforward; it demands substantial expertise across data engineering, software development, cloud infrastructure, and DevOps practices [13], which is why managed orchestration tooling has become a practical necessity rather than an optional enhancement.

Cloud-native approaches further enhance scalability and operational efficiency by leveraging managed infrastructure and on-demand resources. Tamburri (2020) notes that where MLOps and cloud engineering intersect, a range of technical mechanisms and design patterns emerge that make ML operations considerably more efficient [7]. Similarly, most major public cloud providers now offer dedicated MLOps platforms at varying degrees of maturity, and organisations are increasingly treating cloud infrastructure as their primary operational environment rather than a secondary deployment target [12]. Cloud-native architectures are well-positioned to accommodate this shift, offering on-demand execution of complete ML workflows and representing a future-proof approach to scalable deployment [14]. Platforms such as Kubeflow, an open-source cloud-native tool for building data pipelines and ML workflows, illustrate how these principles are applied in practice [14].

For ML prediction applications specifically, cloud deployment unlocks the full value of the MLOps approach. The benefits of MLOps are most apparent in contexts requiring frequent model retraining and redeployment [10], precisely the conditions that characterise production prediction systems operating on live data. Infrastructure-as-code tooling such as Terraform supports this by ensuring that provisioning remains consistent and reproducible across environments, while managed endpoint services allow trained models to be exposed as real-time APIs without bespoke serving infrastructure [17]. In this project, these principles are applied through Google Cloud Platform, where the prediction API and user-facing UI are packaged as Docker container images, built via Cloud Build, and served as serverless Cloud Run services configured to scale to zero when idle and cap at two instances for cost control [4]. Container images are stored in Artifact Registry, the recommended replacement for the legacy Container Registry [3], and a Pub/Sub-triggered Cloud Function provides the reference implementation for asynchronous batch prediction workloads alongside the synchronous HTTP endpoint. Google Cloud's documentation on MLOps continuous delivery identifies this kind of cloud-native, pipeline-driven architecture, combining containerised serving, managed build pipelines, and event-driven components, as the target design for production ML systems [4]. The result is a deployment architecture in which environment parity between local development and cloud production is maintained throughout, and where individual components can be updated and redeployed independently without disrupting the rest of the system [14].

5) Monitoring, Retraining, and Data Drift

Monitoring represents one of the most critical stages of the MLOps lifecycle. Unlike traditional software systems, machine learning models are susceptible to performance degradation caused by changing data distributions, evolving user behaviour, and shifting business requirements. Once deployed, model performance tends to deteriorate as incoming data evolves away from the distribution on which the model was trained [16]. Garg et al. therefore position monitoring as the mechanism responsible for overseeing the full pipeline, detecting degradation, triggering retraining, and ensuring models continue to operate as intended [16]. This view is reinforced by the observation that real-time performance statistics form the basis for such oversight, and that automated drift detection and treatment remains one of the central unsolved challenges in production ML deployment [17].

A major focus of recent research is the detection and mitigation of data drift and concept drift. Mäkinen et al. argue that ML monitoring must account for biases and drift that emerge over time [10], while Faubel et al. describe drift as a product of changes in both the underlying data and the concepts required to make valid inferences from it [8]. When left unaddressed, distributional shifts of this kind directly erode model accuracy [15]. Modern MLOps architectures therefore incorporate monitoring systems capable of tracking model accuracy, data quality, and operational metrics. When performance falls below defined thresholds, automated retraining can be triggered without manual intervention [8], a process that may itself be initiated by schedulers or continuous drift analysis rather than human oversight [6]. Observability tooling supports this by providing visibility into the behaviour, performance, and health of deployed models in production [20], and future-facing implementations are expected to extend this further with input-distribution-based retraining triggered automatically upon drift detection [17].

Recent studies further emphasise the importance of feedback loops and continuous adaptation. A feedback loop connecting the monitoring system back to the training pipeline is essential for preventing model stagnation over time [21]. Garg et al. suggest that monitoring systems should not only evaluate deployed models but also initiate retraining processes when necessary [16], while Sundaram reports that drift-monitoring components can detect distributional shifts and automatically launch new training pipeline executions when performance thresholds are exceeded [18]. Sustainable MLOps frameworks also recognise broader runtime uncertainties — including drift in data quality, model quality, evolving business needs, and the dynamic environmental context in which models operate [22], reinforcing the need for continuous adaptation throughout the ML lifecycle. Responsible deployment of AI systems further demands that this monitoring be ongoing rather than periodic, as continuous risk monitoring is recognised as a prerequisite for accountable AI in practice [23].

6) Reproducibility, Governance and Reliability

MLOps is not about getting something up and running and then keeping an eye on it. MLOps also helps with some problems, like making sure things work the same way every time following the rules and keeping the system running smoothly. These things are really important because they help people trust the machine learning systems fix them when they break and check what they are doing at any time.

To make sure things work the way every time people use things like keeping track of changes watching what the models

do and writing down what the datasets and performance metrics are. A lot of research says it is an idea to keep a record of what changes were made to the models and the pipeline so companies can do the same experiment again compare different versions of the model and make updates in a consistent way. MLOps and machine learning systems are connected in this way. MLOps helps with the machine learning systems. Improving transparency directly supports these goals, aligning with MLOps practices such as reproducibility and versioning [23], and well-implemented pipelines can achieve a standard where the entire process is transparent, reliable, and reproducible end-to-end [6]. In this project, reproducibility is maintained through pinned dependencies, fixed random seeds, sklearn pipelines that capture preprocessing as part of the model artefact, and MLflow experiment tracking that logs parameter grids, chosen hyperparameters, and held-out test metrics for every run [4].

Governance focuses on transparency, accountability, and oversight of machine learning systems. Research highlights the value of model versioning, audit logs, and explainability artefacts in supporting responsible AI deployment [21]. Faubel et al. further suggest that model explanations can act as plausibility checks for both models and datasets, helping identify unexpected behaviour and reducing technical debt throughout the ML lifecycle [8]. Trustworthy AI development more broadly requires that monitoring and governance mechanisms are treated as first-class components of the MLOps pipeline rather than afterthoughts [21].

Reliability is achieved through standardised deployment frameworks, fault-tolerant architectures, and automated operational processes. Poloskei notes that cloud-native architectures improve system robustness through self-healing mechanisms that automatically replace failed components [14]. Similarly, automated validation, monitoring, and retraining processes help maintain model performance as data and operating conditions evolve [15]. The automation of these processes, whether triggered by schedulers or drift analysis, reduces reliance on manual intervention and limits the risk of undetected degradation in production [6].

While reproducibility and reliability are widely discussed in the literature, governance remains comparatively underexplored. Most studies focus on automation and deployment efficiency, while fewer address accountability, compliance, and long-term oversight of machine learning systems [10]. This represents an important area for future MLOps research.

7) Evaluation Metrics

Effective evaluation of an MLOps pipeline requires metrics across two distinct layers: model performance metrics that assess predictive quality, and operational metrics that measure system behaviour in production. Huyen draws this distinction explicitly, noting that a model can perform well on offline benchmarks while failing in deployment due to latency, throughput, or infrastructure constraints [2].

a) Model Performance Metrics

RMSE (Root Mean Squared Error) is the square root of the average squared difference between predicted and actual values:

$$RSME = \sqrt{\frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2} \quad (1)$$

Equation (1) computes RMSE by first taking the difference between each predicted value $\hat{y}_i$ and its corresponding actual value y_i , squaring each difference to remove sign and amplify larger errors, averaging those squared differences across all n samples, then taking the square root to return the error to the original unit of the target variable. Huyen recommends RMSE for regression tasks where large prediction errors carry disproportionate cost [2], which applies here given that a substantially wrong exam score prediction is more harmful than a marginal one. In this project RMSE is tracked across every MLflow run and serves as the cross-validation selection criterion via GridSearchCV.

MAE (Mean Absolute Error) takes the average of the absolute differences between predictions and actuals:

$$MAE = \frac{1}{n} \sum_{i=1}^n |\hat{y}_i - y_i| \quad (2)$$

Equation (2) computes MAE by taking the absolute difference between each prediction $\hat{y}_i$ and actual value y_i , which removes sign without squaring, then averaging across all n samples. Because no squaring occurs, a large error contributes to MAE in direct proportion to its size rather than quadratically. Huyen notes that the choice between MAE and RMSE should reflect the cost structure of errors in the specific application [2]: where the penalty for a large error is not meaningfully worse than for a small one, MAE is the more interpretable and robust choice. Both are reported in this project to give a complete picture of error distribution.

R² (Coefficient of Determination) measures the proportion of variance in the target variable explained by the model:

$$R^2 = 1 - \frac{\sum_{i=1}^n (\hat{y}_i - y_i)^2}{\sum_{i=1}^n (\bar{y} - y_i)^2} \quad (3)$$

Equation (3) computes R² by dividing the residual sum of squares, the total unexplained error from the model, by the total sum of squares, which is the variance that would remain if the model simply predicted the mean $\bar{y}$ for every sample. Subtracting this ratio from 1 gives the proportion of variance the model accounts for. A value of 1.0 indicates perfect prediction; 0 indicates the model performs no better than the mean baseline; negative values indicate it performs worse. Huyen emphasises that no single metric should be used in isolation [2], R² provides contextual scale that RMSE alone does not, since the same RMSE value means something different depending on the variance of the target.

pred_mean records the arithmetic mean of all predictions produced during an evaluation window.

$$pred_mean = \frac{1}{n} \sum_{i=1}^n \hat{y}_i \quad (4)$$

Equation (4) averages all predicted values $\hat{y}_i$ across n predictions in the monitoring window. On its own pred_mean is not a performance metric, but as a monitoring signal it is useful precisely because it does not require ground truth labels: a sustained upward or downward shift in pred_mean relative to the training period can indicate label drift or a changed input population before RMSE has had time to degrade enough to trigger an alert. Huyen identifies output distribution monitoring as a lightweight early-warning

mechanism suited to production settings where labels arrive with a lag [2].

CV scores refer to the per-fold performance estimates produced during k -fold cross-validation, where the dataset is partitioned into k equally sized folds and the model is trained and evaluated k times, each time holding out a different fold as the validation set.

$$CV_{score} = \frac{1}{k} \sum_{i=1}^k score_i \quad (5)$$

Equation (5) averages the evaluation metric, negative RMSE in this project, across all k folds to produce a single cross-validation estimate. The individual fold scores $score_i$ are also informative in their own right: high variance across folds indicates the model is sensitive to the specific data partition, while low variance indicates stable generalisation. Huyen argues that cross-validation is essential for reliable model selection on moderately sized datasets where a single train/test split may produce estimates that reflect the particular composition of that split rather than genuine model quality [2]. In this project, 5-fold cross-validation is run inside GridSearchCV and the parameter configuration with the best mean negative RMSE is selected.

b) Drift Metrics

KS-test p-value (Kolmogorov–Smirnov test) is a non-parametric statistical test that measures whether two samples are drawn from the same distribution. The test statistic D is the maximum absolute difference between the empirical cumulative distribution functions of the two samples:

$$D = \sup_x |F_{ref}(x) - F_{cur}(x)| \quad (6)$$

Equation (6) computes the KS statistic D as the maximum absolute vertical distance between the empirical cumulative distribution function of the reference sample F_{ref} , the training data distribution, and the current production sample F_{cur} , taken across all possible values of x . The larger D is, the more the two distributions diverge. The p-value then expresses the probability of observing a gap at least as large as D if both samples genuinely came from the same distribution: a low p-value (conventionally below 0.05) rejects that assumption and flags the feature as drifted. Huyen discusses two-sample KS tests as a standard method for detecting covariate drift in continuous input features [2]. In this project, `study_hours_per_day` and `social_media_hours` both produced low p-values during the recorded drift event, confirming the monitoring layer was functioning as intended.

share_drifted_features is a summary statistic derived from the per-feature KS test results, expressing the fraction of monitored features for which drift was detected.

$$shared_drifted = \frac{\text{number of features with } p < 0.05}{\text{total number of monitored features}} \quad (7)$$

Equation (7) divides the count of features that failed the KS test, those whose p-value fell below the significance threshold, by the total number of features under monitoring. A single drifted feature may have limited downstream impact depending on its importance to the model, but a high share_drifted value signals broad distributional shift across the input space, indicating the incoming data population has

moved substantially away from the training distribution. Huyen recommends aggregating feature-level drift signals into a pipeline-level health indicator of this kind [2], as it provides a single actionable threshold that can trigger retraining without requiring manual inspection of each feature individually.

c) Operational Metrics

Cold-start latency measures the time from a request arriving at a Cloud Run instance that has scaled to zero until the first response is returned. This encompasses container initialisation, dependency loading, and model deserialisation. Huyen distinguishes between latency measured at the infrastructure level and latency experienced by the end user [2], and flags cold-start as a characteristic cost of serverless deployment that must be weighed against the cost savings of scale-to-zero. In this project, Cloud Run services are configured with a minimum of zero instances, meaning cold-start latency is a real user-facing cost for low-traffic periods.

API and UI image size determines how long Cloud Build takes to push an image and how quickly a new instance can pull and start it, directly affecting both cold-start latency and CI/CD pipeline duration. Huyen notes that container image size is a practical operational concern in production ML systems, where large images, driven by heavyweight dependencies such as PyTorch or TensorFlow, can significantly slow deployment pipelines [2]. Tracking image size across builds is therefore a proxy metric for deployment efficiency and forms part of the CI/CD health picture in this project.

II. METHODOLOGY

A. System Architecture

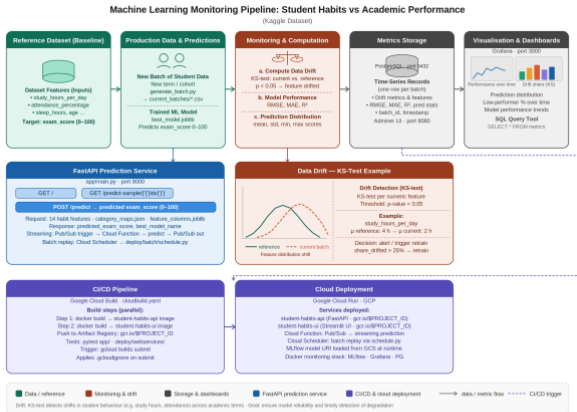


Figure 1 - System Architecture (Generated Image using Claude)

Figure 1 depicts the report’s system architecture, the pipeline follows a directed flow from data and experimentation through to live serving. A training notebook ingests the student habits dataset, runs model selection and hyperparameter tuning, and logs every experiment, parameter grids, chosen hyperparameters, and held-out metrics, to MLflow. The best-performing model is serialised as a joblib artefact alongside its preprocessing companions and pushed to Google Cloud Storage, where it is consumed by the serving layer. The repository is structured around four independently

deployable components: the synchronous prediction API (app/), the demo UI (streaming/ui/), the event-driven streaming function (streaming/function/), and the local monitoring stack (monitoring/). Each component has a single responsibility and its own Dockerfile or runtime configuration, meaning any one of them can be updated and redeployed without touching the others. This container-first approach ensures that the artefact running locally during development is identical to what runs in production, eliminating environment-specific failures. For serving, Cloud Run and Cloud Functions were chosen over a long-running VM or Kubernetes cluster. Given the low and unpredictable traffic of a demo system, the scale-to-zero behaviour of serverless infrastructure keeps costs bounded, instances spin up on request and release when idle, with a hard cap of two concurrent instances. External dependencies are kept minimal: Pub/Sub handles asynchronous batch prediction requests between the streaming function and downstream consumers, Artifact Registry stores built container images, and Cloud Build provides the managed image build pipeline. As shown in Figure 1, data flows left to right from the reference dataset and production inputs through training, serving, monitoring, and visualisation, with the CI/CD and cloud deployment layers operating beneath as the operational backbone.

B. Modelling and Reproducibility

The Kaggle "Student Habits vs Academic Performance" dataset by jayaantanaath contains 1,000 student records across 16 columns: one identifier (student_id), 14 features, and the continuous target variable exam_score (0–100 scale). The features span numerical variables: age, study_hours_per_day, social_media_hours, netflix_hours, attendance_percentage, sleep_hours, exercise_frequency, and mental_health_rating — and six categorical variables: gender, part_time_job, diet_quality, parental_education_level, internet_quality, and extracurricular_participation. Two engineered features were derived: total_screen_time (the sum of social media and Netflix hours) and study_to_leisure_ratio (study hours divided by total screen time), capturing how a student allocates time between academic effort and leisure. The only data quality issue was missing values in parental_education_level (91 rows, 9.1%), addressed through mode imputation.

As shown in Figure 2, exam_score is approximately normally distributed with a mean of 69.6 and a median of 70.5, ranging from 18.4 to 100 with a small number of low-end outliers visible in the box plot. The numerical features in Figure 3 are all near-symmetric with low skew values across the board, indicating the distributions are well-behaved and require no log transformation prior to modelling.

The feature correlation ranking in Figure 4 reveals a clear hierarchy of predictive signal. study_hours_per_day is by far the strongest predictor at $r = 0.83$, and the corresponding scatter plot in Figure 5 confirms this as a near-linear relationship across the full score range. study_to_leisure_ratio follows at $r = 0.50$, reflecting that the balance between study and screen time matters beyond raw study hours alone. mental_health_rating contributes a moderate positive relationship at $r = 0.32$, while total_screen_time, social_media_hours, and netflix_hours all show weak negative correlations. age, attendance_percentage, sleep_hours, and exercise_frequency have negligible

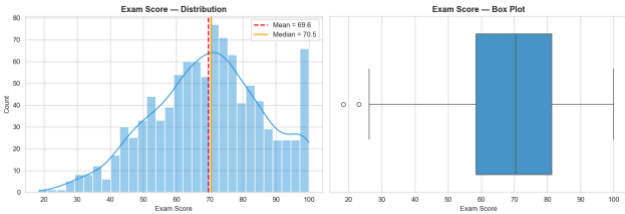


Figure 2 - Exam score distribution and boxplot

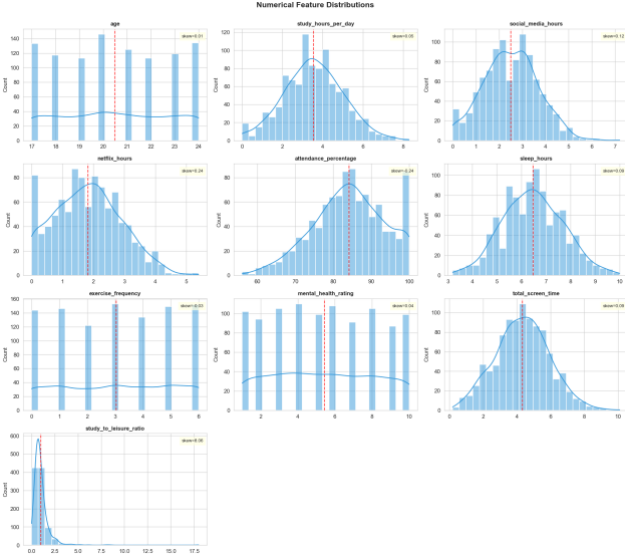


Figure 3 - Numerical features distribution

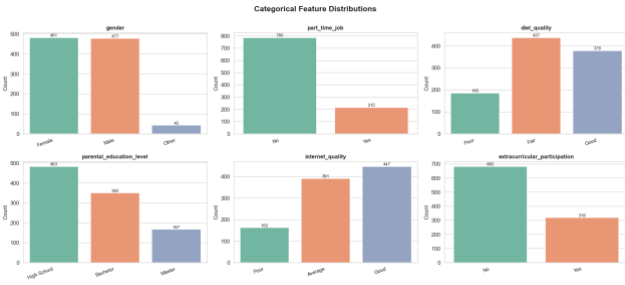


Figure 4 - Categorical feature distribution

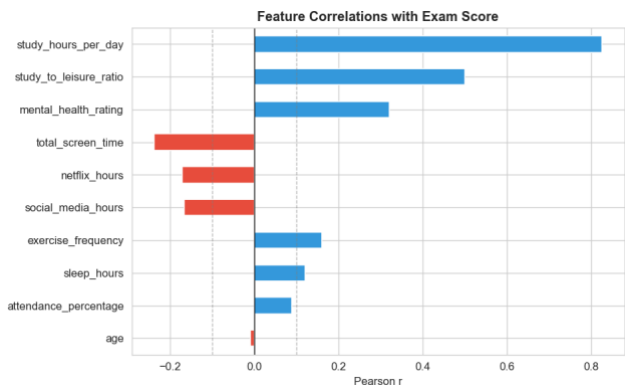


Figure 5 - Feature correlations with exam score

correlations with exam score, suggesting limited individual predictive value.

As previously stated, the dataset has a combination of numerical and categorical features, the issue with the numerical features is that the model cannot process this and

needed to be encoded so that the computer works. During preprocessing the categorical features were encoded using a deterministic category_map e.g. male = 0, female = 1, and other = 2. The feature parental_education_level suffered from a lot of missing values, to combat this the NaN values were replaced with the mode education level, high school, as it is a small dataset which is used to show the MLOps pipeline. If the focus was on the model, this is not good practice and those missing parental educational would be removed instead. The numerical features were standardised using sklearn StandardScaler so that larger features like attendance do not monopolise the model's learning process.

After preprocessing, the data is split 80-20 for a train test split which is standard practice for ML model development. Five hyperparameter-tuned base models (Ridge Regression [24], Lasso [25], K-Nearest Neighbours [26], Random Forest [27], XGBoost [28]) and one VotingRegressor ensemble were trained on an 800-row training set and evaluated on a 200-row held-out test set under the student-habits-performance MLflow experiment. Each base model was tuned via 5-fold GridSearchCV [29]; the VotingRegressor [30] was constructed by unweighted averaging of the five tuned estimators.

VotingRegressor. The VotingRegressor ensemble was constructed from the five hyperparameter-tuned base estimators (Ridge, Lasso, K-Nearest Neighbours, Random Forest, and XGBoost) using scikit-learn's VotingRegressor with no weights specified, which produces an unweighted arithmetic average of the five constituent predictions. The ensemble was trained on the same 800-row training partition and evaluated on the same 200-row held-out test partition as the individual base models, with results logged to MLflow as a separate run under the student-habits-performance experiment. No cross-validation was performed on the ensemble itself, since its constituents had already been individually tuned via GridSearchCV. The VotingRegressor was included specifically to test whether ensembling the five base estimators improves over the strongest individual model; the empirical result, reported in Model Performance section, is that on this dataset it does not, because KNN's substantially weaker predictions degrade the unweighted average.

Hyperparameter Tuning. Hyperparameter tuning was performed independently for each of the five base models using scikit-learn's GridSearchCV. Each model was paired with a model-specific parameter grid: Ridge over $\alpha \in \{0.1, 1.0, 10.0, 100.0\}$; Lasso over $\alpha \in \{0.001, 0.01, 0.1, 1.0\}$; K-Nearest Neighbours over the product of $n_neighbors \in \{5, 10, 15, 20\}$ and $weights \in \{uniform, distance\}$; Random Forest over the product of $n_estimators \in \{100, 200\}$, $max_depth \in \{5, 10, 20\}$, and $min_samples_split \in \{2, 5\}$; and XGBoost over the product of $n_estimators \in \{100, 200\}$, $max_depth \in \{3, 5\}$, and $learning_rate \in \{0.05, 0.1\}$. Each grid was searched exhaustively under 5-fold KFold cross-validation on the 800-row training set, scored on negative root mean squared error. The configuration with the highest mean score across folds was retained: Ridge selected $\alpha = 1.0$; Lasso selected $\alpha = 0.1$; K-Nearest Neighbours selected $n_neighbors = 10$ with distance weighting; Random Forest selected $n_estimators = 200$, $max_depth = 20$, $min_samples_split = 2$; and XGBoost selected $n_estimators = 100$, $max_depth = 3$, $learning_rate = 0.1$. The VotingRegressor was not separately

tuned; it was constructed as an unweighted average of the five already-tuned base estimators.

The parameters and hyperparameters of each model (RMSE / MAE / R^2) is displayed on MLflow's UI. The benefit

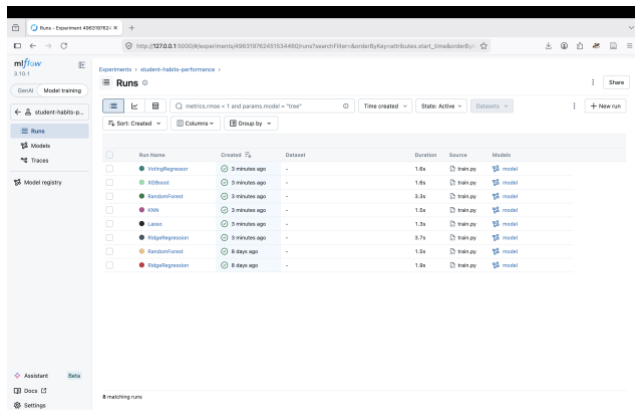


Figure 6 - MLflow UI depicting the list of tested models

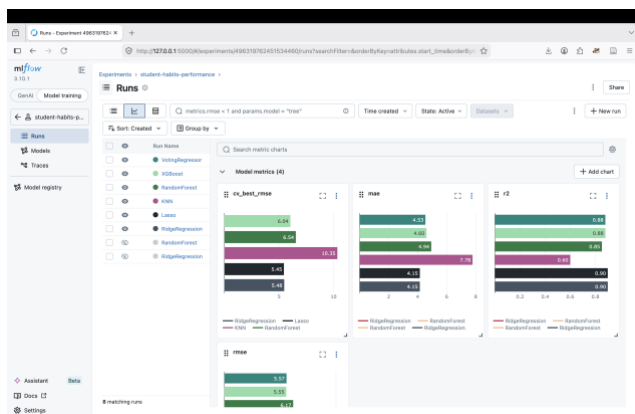


Figure 7 - MLflow website comparing the hyperparameters of each model

of using MLflow is each experiment of a model is saved in the /mlruns folder which can then be seen using the MLflow UI, it provides a full experiment lineage allowing (Figure 6) the user to go back to parameters that resulted in more accurate hyperparameters.

C. CI / CD and Automation

CI / CD test the pure prediction logic in isolation, without standing up the FastAPI app. They use mock fixtures so the suite runs without the trained model present. The CI / CD pipeline occurs in GitHub where automated tests are set up in the repository so that when an action occurs; a push, a set of tests are automated trigger an image to be built which is then deployed, a downloadable and reproducible container image (Figure 8). The 22 tests during CI can be separated into four key categories: smoke and metadata (a quality gate), functional / happy-path prediction (makes sure the prediction returns a sensible result on valid input), score clipping (to make sure the predictions are within the valid 0–100 range), and input validation (making sure all malformed, missing, out-of-range, wrong-type, or unknown-category inputs are all rejected). Whether the push is accepted, passing the tests, can be seen in the actions GitHub tab (Figure 8).

The Cloud Build pipeline is defined in a single cloudbuild.yaml at the project root, which builds both Cloud

Run images, the prediction API and the demo UI, in parallel and pushes both to Artifact Registry. Parallel execution is achieved by setting waitFor: ['-'] on the second build step, which instructs Cloud Build not to wait for any prior step; together with a dual images declaration, this produces both `gcr.io/$PROJECT_ID/student-habits-api:latest` and `gcr.io/$PROJECT_ID/student-habits-ui:latest` from a single gcloud builds submit invocation. The decision to consolidate both image builds into one configuration, rather than maintaining a separate cloudbuild.yaml per service, reflects the project's actual deployment cadence: the API and UI are rebuilt together whenever the deployed system changes, so per-service configurations would have introduced duplication without matching how the pipeline is used in practice. The consolidation also reduces the surface area for the kind of path-related build errors documented in §V.C. Within the CI/CD pipeline itself, automation coverage is partial: the GitHub Actions test suite runs automatically on every push to main, but image builds are triggered manually by the developer running `gcloud builds submit`, and deployment to Cloud Run is likewise triggered manually via `gcloud run deploy`. By the maturity model in [4], this places the system between Levels 0 and 1, automated continuous integration but manual continuous delivery, rather than a fully Level 1 pipeline in which the build and deploy stages are themselves triggered automatically. Mention testing approach: unit tests cover the prediction core logic, integration tests cover the API endpoints, and tests are designed to run without the actual model present (mock fixtures) so CI doesn't need access to the trained artefacts

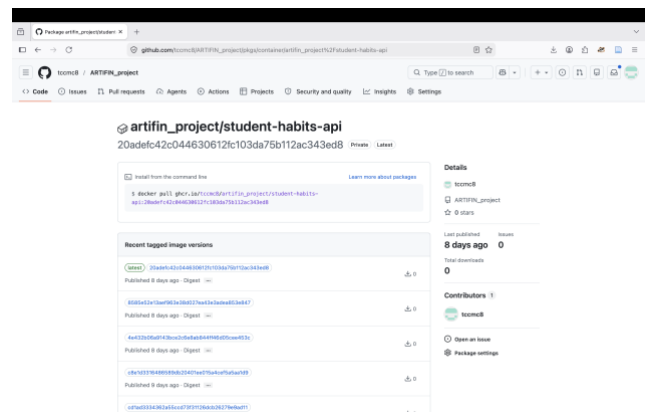


Figure 8 - Container image on GitHub and its updates

D. Google Cloud Deployment

The deployed system runs in the Google Cloud us-central1 region. The choice of a single region was driven by three factors: cost (us-central1 sits at the lower end of GCP's pricing tiers), the consistent availability in that region of every managed service required by the pipeline (Cloud Run, Cloud Functions 2nd generation, Pub/Sub, and Artifact Registry), and the elimination of inter-region network latency by co-locating all components. The two Cloud Run services, the prediction API (Figure 9) and the demo UI (Figure 10), are deployed with identical scaling configuration: `--min-instances=0` to scale to zero between requests and eliminate idle cost, `--max-instances=2` to cap concurrent instances and bound maximum spend, and `--allow-unauthenticated` to expose the public demo endpoint without an authentication layer. The streaming Cloud Function provides the asynchronous counterpart to the synchronous API: it is Pub/Sub-triggered, subscribes to the student-habits-requests

topic, performs inference on the incoming CloudEvent payload, and publishes the result to the student-habits-predictions topic. At cold start, the function lazy-loads the trained model and preprocessing artefacts (category_maps.json, feature_columns.joblib) from Google Cloud Storage, with their GCS paths supplied through environment variables (MODEL_URI, CATEGORY_MAPS_URI, FEATURE_COLUMNS_URI) and the project identifier through GCP_PROJECT. No credentials are baked into the container image, and the function can be redeployed independently of changes to the underlying artefacts. For the live demo this streaming path was bypassed: the UI calls the synchronous /predict endpoint of the prediction API directly via HTTP, because the latency of an end-to-end Pub/Sub round-trip is not suited to a real-time form-and-result interaction. The streaming function therefore remains as the reference implementation for batch and asynchronous workloads, where Pub/Sub's decoupling and rate-control properties matter more than per-request latency. Container images are stored in Artifact Registry under the gcr.io/PROJECT-ID/... path, the same registry path Cloud Build writes to automatically when the images declaration described in §III.C is processed, eliminating the need for any inter-stage image transfer. The boundary between this section and §III.C is the gcloud run deploy command itself: §III.C describes how the images are built and pushed by the pipeline, while this section describes how those pushed images are configured at runtime when the deploy command is invoked.

E. Monitoring and Maintenance

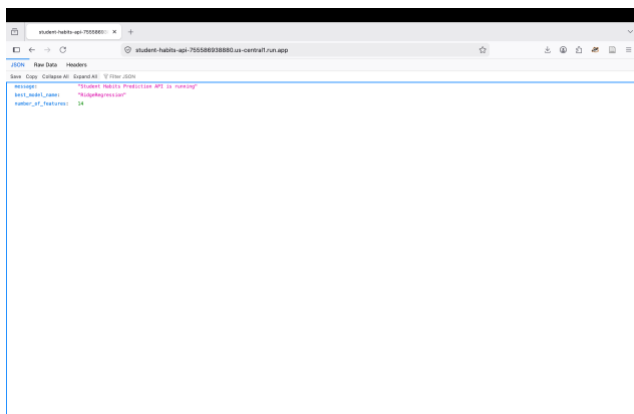


Figure 9 - Prediction API (machine readable service)

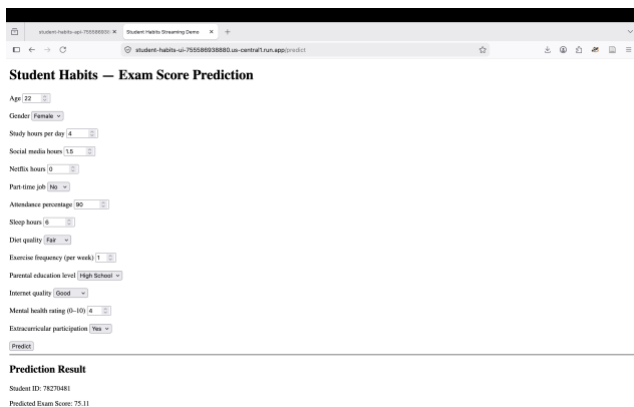


Figure 10 - Demo UI (Human-facing)

The monitoring stack is defined in monitoring/docker-compose.yaml and orchestrates five containerised services:

Postgres 16 (student-habits-monitoring-db) acts as the persistence layer for both prediction logs and batch drift metrics; an MLflow server (running on port 5001 to avoid a port conflict with macOS AirPlay) tracks experiments and acts as the model registry, with its backend store also in Postgres; Grafana (grafana-oss) reads from Postgres and visualises the monitoring panels; Adminer provides direct database inspection on port 8080; and a containerised instance of the FastAPI prediction service is included for development-time integration testing. Every call to the prediction API writes a record to Postgres containing the request timestamp, the input feature values, the predicted score, and the model name, providing a complete audit trail of production inference. The drift-detection batch job in monitoring/scripts/calculate_metrics.py performs a two-sample Kolmogorov-Smirnov test against the reference training distribution for each of the eight monitored numeric features (age, study_hours_per_day, social_media_hours, netflix_hours, attendance_percentage, sleep_hours, exercise_frequency, mental_health_rating), flagging any feature whose p-value falls below 0.05 as drifted, and computes per-batch RMSE and R^2 against a held-out evaluation set; the per-feature p-values, the count and share of drifted features, and the regression metrics are written back to Postgres for Grafana to query. The retraining loop, though not fully automated, follows a defined path: drift detection produces a flagged batch record visible in Grafana, the developer is notified through the dashboard, src/train.py is re-run manually, a new MLflow run is logged, and the new best model is promoted by replacing models/best_model.joblib and redeploying the API. This semi-automated loop is the gap between Levels 1 and 2 of the maturity model in [4].

The Performance Summary panel (Figure 11) reports an aggregate production RMSE of 7.61 over a 30-day window, against a held-out test RMSE of 5.10 reported in Model Performance section a 49% inflation in error that is attributable to deployment on shifting inputs rather than to any code defect, infrastructure failure, or labelling error, and that quantifies the cost of running a static model in a non-stationary environment. The Batches With Any Drift Detected panel (Figure 12) enumerates three events: sleep_hours drifted on 2026-04-21 (batch 38d392dc); study_hours_per_day and social_media_hours both drifted on 2026-04-28 (batch fb8a8203); and the same two features drifted again on 2026-06-03 (batch e1a071c6). The latter two events are the more consequential, because the standardised Ridge coefficients reported in Model Performance section identify study_hours_per_day (+14.14) and social_media_hours (-3.15) as the two most influential features in the model — together accounting for more total coefficient magnitude than the other twelve features combined. Drift in these features is therefore not arbitrary: it is drift in precisely the inputs to which the model is most sensitive, and the Prediction Spread panel (Figure 13) shows the operational consequence directly. Prior to the 28 April event the mean prediction tracked the training distribution at approximately 70 and the maximum at approximately 110; following the event the mean jumped to approximately 95 and the maximum to approximately 143, and during the same window the batch-level RMSE rose to 21.80 and R^2 fell to -0.64. The narrow feature subset flagged across events also serves as evidence against false-positive detection: a KS test at $p < 0.05$ across the eight monitored features has approximately a 34% per-batch probability of flagging at least one feature spuriously when no genuine drift

exists ($1 - 0.95^8 \approx 0.34$), but drift was observed on only 3 of 13 batches and on a consistent subset of features each time, which is the signature of genuine distributional shift rather than statistical noise.

What the monitoring layer demonstrably catches is covariate drift in observed numeric features. What it does not

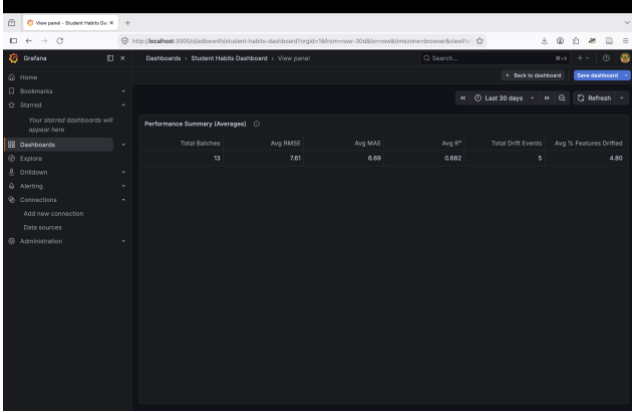


Figure 11 - Grafana: Performance Summary panel

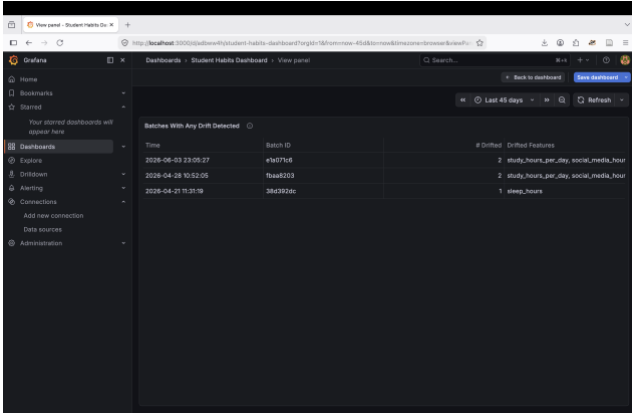


Figure 12 - Grafana: Batches With Any Drift Detected pane

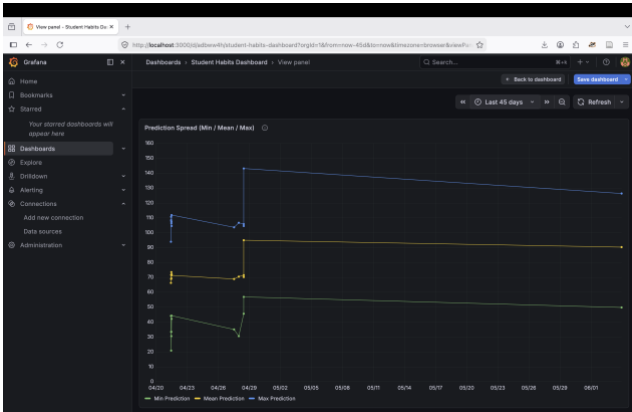


Figure 13 - Grafana: Prediction Spread (Min / Mean / Max) panel

catch is equally important to name. Concept drift — where the relationship between features and target shifts while input distributions look stable — is invisible to the present setup, because detecting it requires labelled feedback (actual exam scores arriving with a lag) that the system does not currently consume. Drift in unobserved features, for example the emergence of short-form video as a procrastination vector not captured by either `netflix_hours` or `social_media_hours`, is invisible by construction; only feature engineering and

retraining on updated data can address this category. Beyond the synthetic drifts present in the evaluation set, several plausible drifts are foreseeable on the current feature set. `social_media_hours` is likely to drift upward over multi-year horizons consistent with the secular rise in student digital consumption. `netflix_hours` is likely to drift downward as streaming consumption fragments away from Netflix to other platforms — a case where the measured feature drifts while the underlying behaviour it was meant to proxy (recreational screen time) does not, undermining the feature's predictive validity without necessarily producing a corresponding KS-test signal. `attendance_percentage` is a candidate for both covariate drift, as policies around hybrid lecture delivery evolve, and concept drift, as recorded lectures decouple physical attendance from engagement and weaken the feature's predictive relationship with `exam_score` while leaving its measured distribution intact. `mental_health_rating` baselines have shifted measurably in student populations post-2020 and are likely to continue evolving. `internet_quality` is likely to drift towards higher categories as infrastructure improves, eroding the variance the model can use to discriminate between students. That the monitoring stack runs locally rather than in managed cloud services is a deliberate scope decision: hosting Postgres, MLflow, and Grafana on Cloud SQL and equivalent managed services would have added recurring infrastructure cost without changing the analytical findings of this report.

III. RESULTS

A. Model Performance

Model	Best hyperparameters	CV best RMSE	Test RMSE	Test MAE	Test R ²
Ridge	$\alpha = 1.0$	5.48	5.10	4.15	0.899
Lasso	$\alpha = 0.1$	5.45	5.10	4.15	0.898
XGBoost	depth = 3, lr = 0.1, n = 100	6.04	5.55	4.60	0.880
Random Forest	depth = 20, n = 200	6.54	6.17	4.94	0.852
KNN	k = 10, distance-weighted	10.35	10.13	7.78	0.600
VotingRegressor	unweighted average of all five base models		5.57	4.53	0.879

The ranking on the held-out test set is clear: Ridge and Lasso are statistically indistinguishable at the top, with test RMSEs of 5.098 and 5.103 and test R² values of 0.8986 and 0.8985 respectively. XGBoost ranks third (RMSE 5.55, R² 0.880), the VotingRegressor ranks fourth (RMSE 5.57, R² 0.879), Random Forest ranks fifth (RMSE 6.17, R² 0.852), and KNN trails substantially (RMSE 10.13, R² 0.600). Cross-validation stability is good: the test RMSEs lie within 0.5 RMSE units of the CV best RMSEs across all five tuned base models, and the relative ranking is preserved exactly between CV and test. This consistency indicates the observed ranking is a property of the data rather than an artefact of the train/test split.

The empirical result, regularised linear regression outperforming gradient boosting and random forests, reflects the linear nature of the dominant predictive signal in the

dataset. Inspection of the Ridge model's coefficients (computed on standardised features) confirms this directly: `study_hours_per_day` carries a coefficient of +14.14, more than twice the magnitude of any other feature. The next four largest, in order, are `mental_health_rating` (+5.56), `social_media_hours` (-3.15), `exercise_frequency` (+2.65), and `netflix_hours` (-2.53); the remaining nine features each have absolute coefficients below 2.5, and four (gender, parental education level, internet quality, extracurricular participation) have effects indistinguishable from zero. This pattern is consistent with the exploratory analysis finding that `study_hours_per_day` had the strongest Pearson correlation with `exam_score` ($r = 0.83$) in the raw data. With one strong linear predictor and a small number of weaker ones, a regularised linear model exploits the structure efficiently; tree-based and instance-based methods pay a generalisation cost for flexibility the data does not require.

The VotingRegressor's underperformance relative to Ridge has a specific cause. As an unweighted average across all five base estimators, the ensemble inherits a fixed share of KNN's predictions; KNN's R^2 of 0.600 is dramatically lower than the others' (≥ 0.852), so averaging its outputs degrades overall ensemble accuracy. The ensemble's R^2 of 0.879 sits between the simple mean of the five base R^2 values (0.826) and the strongest constituent (Ridge at 0.899), consistent with averaging weakly diverse but unevenly accurate estimators. Ensemble averaging improves performance when its constituents contribute either accurate or genuinely independent errors; here KNN contributes predictions whose errors are both larger than and correlated with those of the other models, and the result adds noise rather than diversity.

B. Deployment Metrics

Container image sizes for the two Cloud Run services were measured via Artifact Registry: the API image is 726 MB, containing the FastAPI runtime, scikit-learn, XGBoost, numpy, scipy, and the trained model artefact; the UI image is 50.2 MB, containing only the FastAPI runtime and the Jinja2 templating layer. The combined deployed footprint is 776 MB. An earlier API build of 307.3 MB, predating the inclusion of XGBoost and the expanded model comparison, demonstrates that the additional model families approximately doubled the API image size. A single cold-start request to the `/predict` endpoint, issued after the service had scaled to zero, completed in 15.92 s; of this, 113 ms was network setup (DNS 18 ms, TCP 26 ms, TLS 69 ms) and the remaining 15.81 s was server-side container initialisation, library import, and model deserialisation. Ten successive warm requests to the same endpoint produced latencies between 174 and 192 ms, with a median of 177 ms, a mean of 179 ms, and a range of 18 ms, indicating predictable performance under repeated requests. The cold-start figure is a direct consequence of the `min-instances=0` configuration, which trades first-request latency for the elimination of idle infrastructure cost and is appropriate for the low-traffic profile of a demonstration deployment. Cloud Build wall-clock duration and end-to-end deploy time (gcloud builds submit followed by gcloud run deploy) were not captured in the present measurement run. The streaming Cloud Function is retained as a reference implementation only and was not exercised for latency measurement.

C. CI / CD Metrics

The GitHub Actions CI/CD workflow was triggered on every push to the main branch during development. Across the project's lifetime, 13 workflow runs were executed, all on main: 7 succeeded and 6 failed, giving an overall success rate of approximately 54%. Run durations were strongly bimodal: the six failed runs all completed in 1 m 10 s – 1 m 28 s (mean 1 m 21 s), while the seven successful runs all completed in 2 m 43 s – 3 m 6 s (mean 2 m 56 s). This separation reflects the workflow's structure: failed runs terminated early at the docker build step in the CD job, whereas successful runs proceeded through the entire build and push phase. The CI portion executes a pytest suite of 22 tests, divided between the prediction-core unit tests (deploy/webservices/test_prediction_core.py) and the FastAPI integration tests (app/test_api.py); the suite completes in 1.08 s locally. The six failures were not spurious. Run #7 (Simplify clip_score function) failed when a refactoring of the score-clipping logic broke a unit test, prompting the immediate corrective commit in run #8 (Restore clip_score safety check), which passed. Runs #10–#13 form a connected sequence triggered by the consolidation of Dockerfiles: the workflow's docker build step continued to reference the removed root Dockerfile, and even after the path was corrected in run #12, the build failed on COPY models/ because the model artefacts were excluded from version control via .gitignore and were therefore absent from the GitHub-hosted runner's checkout. This failure mode does not affect the Cloud Build pipeline, which uses .gcloudignore to explicitly include models/ in its build context. In each case, the CI workflow surfaced the regression before the associated change could reach a production deployment. Cloud Build duration with and without layer caching was not captured in the present measurement run. A 54% appears low but it is beneficial to the repository as the pipeline fails fast and succeeds slow, every failure was caught before it could reach production and cause any damage, which is the purpose of the CI / CD pipeline.

IV. DISCUSSION

A. Strengths

End-to-end pipeline coverage: the project implements the full MLOps lifecycle, data ingestion, model training with experiment tracking, containerised deployment, CI/CD automation, and drift monitoring, rather than treating any one stage in isolation.

Reproducibility is built in: pinned dependencies, fixed random seeds, sklearn pipelines that bundle preprocessing with the model artefact, and MLflow logging of all run's parameter grids and metrics give the project strong experiment lineage and reproducibility.

Container-first, cloud-native architecture: every runtime component is shipped as a Docker image, maintaining environment parity between local development and GCP. The separation of the four deployable components (prediction API, demo UI, streaming function, monitoring stack) means each can be updated independently.

Drift detection functioned: the monitoring layer caught a real degradation event, features `study_hours_per_day` and `social_media_hours` shifted, causing RMSE to climb to 21.80

and R^2 to fall to -0.64. This validates that the monitoring design works as intended, not just in theory.

Cost-conscious cloud design: scale-to-zero Cloud Run configuration with a two-instance cap demonstrates practical awareness of infrastructure costs, which is a genuine operational concern in production MLOps deployments.

B. Weaknesses

Retraining is not automated: drift detection triggers a notification or manual intervention rather than an automatic retraining run. This places the system at MLOps maturity Level 1 rather than Level 2, the feedback loop from monitoring back to training is not closed.

Image builds and deployment remain manual: CI tests run automatically on every push, but gcloud builds submit and gcloud run deploy are triggered manually. The pipeline is not fully continuous.

Monitoring runs locally, not in the cloud: the Postgres, Grafana, and MLflow monitoring stack is defined in a local docker-compose.yaml. This is acknowledged as a cost and scope decision, but it means production monitoring is not co-located with the deployed services.

Dataset limitations: the model is trained on a 1,000-row Kaggle dataset with synthetic student habits data. This limits the generalisability of any conclusions drawn about model performance or drift behaviour.

Governance is thin: there are no audit logs, access controls, or explainability artefacts attached to the deployed model. The project covers reproducibility and reliability but leaves governance largely unaddressed.

C. Lessons Learned

The gap between Level 0 and Level 1 maturity is the hardest to close: automating tests is straightforward; automating the build, deployment, and retraining loop requires coordinating cloud credentials, trigger logic, and pipeline orchestration across multiple services, the complexity compounds quickly.

Environment parity is worth investing in early: using Docker from the start meant the same artefact ran locally and on Cloud Run without modification. Without this, cloud-specific failures would have been much harder to debug.

Monitoring is only useful if it can trigger action: detecting the drift event was a success, but without an automated retraining trigger, detection alone does not close the loop. Monitoring design should be planned alongside retraining strategy, not added afterwards.

Separation of concerns between components simplifies deployment: keeping the prediction API, UI, streaming function, and monitoring stack as independent deployable units meant failures and updates in one component did not cascade across the system.

Synthetic datasets constrain operational learning because the dataset does not reflect real-world distributional complexity, the drift detected was relatively clean and predictable. A noisier, real dataset would stress-test the monitoring and retraining pipeline more meaningfully.

V. CONCLUSION

This paper presented an end-to-end MLOps pipeline applied to a student exam-score prediction task, framing the prediction problem as a vehicle for demonstrating the

research-to-production lifecycle rather than as the primary contribution. Five hyperparameter-tuned base models and one VotingRegressor ensemble were compared under MLflow experiment tracking, deployed as containerised services on Google Cloud Run, integrated with a GitHub Actions CI/CD pipeline, and monitored for data drift through a Postgres-and-Grafana stack. Ridge regression and Lasso were statistically indistinguishable at the top of the empirical ranking, with held-out test R^2 values of 0.899 and 0.898 respectively, while KNN trailed substantially at $R^2 = 0.600$; inspection of the Ridge coefficients confirmed that the dataset's predictive signal is concentrated in a single, linear feature, `study_hours_per_day`, explaining why complex models did not improve over regularised linear regression. On the operational side, the deployed services produced a median warm latency of 178 ms against a 15.92 s cold-start cost, the CI workflow ran 22 tests in 1.08 s, and the drift-detection batch job correctly identified a covariate drift event in `study_hours_per_day` and `social_media_hours`, with RMSE rising to 21.80 and R^2 collapsing to -0.64.

These findings demonstrate that integrating individually mature MLOps components, MLflow, FastAPI, Docker, Cloud Build, Cloud Run, GitHub Actions, Postgres, and Grafana — into a coherent pipeline produces a system in which environment parity is preserved from local development through to cloud production, experiments are reproducible from pinned dependencies and logged hyperparameters, and real performance degradation is detected by the monitoring layer rather than left to chance. The implementation does not, however, close the full retraining loop. Drift detection triggers operator notification rather than an automated retraining run; image builds and deployments are issued manually through gcloud builds submit and gcloud run deploy; the monitoring stack runs locally rather than co-located with the production services; and governance components such as audit logs and access controls are largely absent. By the maturity model in [4], this places the system between Levels 0 and 1, with selected Level 2 capabilities but not full continuous training.

Future work should focus on closing the gap between the current implementation and Level 2 maturity. Automating retraining on drift detection, including model registration, validation against a holdout set, and promotion of the new model, would close the feedback loop that monitoring presently leaves open. Migrating the monitoring stack to managed cloud services (Cloud SQL, hosted Grafana, hosted MLflow) would make production observability resilient to the loss of the local environment. Adding GitHub Actions deployment via Workload Identity Federation would convert the present semi-manual deployment chain into a fully continuous pipeline. Beyond automation, evaluating the pipeline on a non-synthetic dataset with longitudinal data and natural distributional complexity would more meaningfully stress-test the drift detection and retraining logic than the 1,000-row Kaggle dataset used here.

AI DISCLOSURE

This project was developed with the assistance of an AI coding and writing assistant (Claude, made by Anthropic). In line with academic guidance on disclosure of generative-AI

tool usage, the following describes the role AI played in this work.

AI assistance was used throughout the development of the codebase for: Debugging and error resolution: diagnosing Python runtime errors, Docker and Cloud Build failures, deployment issues on Cloud Run, and shell / gcloud command misconfigurations. As well as architectural, design and code feedback. All AI-generated code was reviewed, tested, and integrated by the author. All cloud deployments and live runs were executed and verified by the author. Final design decisions and responsibility for the working system rest with the author.

AI assistance was used in preparing the written report for: grammar, spelling, proof-reading, and structural feedback, including critiques and recommendations. The substantive arguments, analytical conclusions, and selection of evidence in the report are the author's own.

REFERENCES

- [1] A. Silaghi, 'Best Google Cloud Run Alternatives in Europe (2026) | DanubeData', DanubeData. Accessed: Jun. 05, 2026. [Online]. Available: <https://danubedata.ro/blog/best-google-cloud-run-alternatives-europe-2026>
- [2] W. C. Huyen and K. Li, *Designing Machine Learning Systems*. Accessed: Jun. 05, 2026. [Online]. Available: <https://learning.oreilly.com/library/view/designing-machine-learning/9781098107956/>
- [3] 'Transition from Container Registry to Artifact Registry in GKE | Google Kubernetes Engine (GKE)', Google Cloud Documentation. Accessed: Jun. 05, 2026. [Online]. Available: <https://docs.cloud.google.com/kubernetes-engine/docs/deprecations/container-registry>
- [4] 'MLOps: Continuous delivery and automation pipelines in machine learning | Cloud Architecture Center', Google Cloud Documentation. Accessed: Jun. 05, 2026. [Online]. Available: <https://docs.cloud.google.com/architecture/mlops-continuous-delivery-and-automation-pipelines-in-machine-learning>
- [5] L. Harisha, 'Hidden Technical Debt in Machine Learning Systems', Medium. Accessed: Jun. 05, 2026. [Online]. Available: <https://lathashreeh.medium.com/hidden-technical-debt-in-machine-learning-systems-27fa1b13040c>
- [6] P. Sobral, R. Teixeira, R. Marques, N. Figueiredo, M. Antunes, and D. Gomes, 'Improving Automotive Aftermarket Forecasting with MLOps', in *2024 11th International Conference on Future Internet of Things and Cloud (FiCloud)*, 2024, pp. 382–389. doi: 10.1109/FiCloud62933.2024.00065.
- [7] D. A. Tamburri, 'Sustainable MLOps: Trends and Challenges', in *2020 22nd International Symposium on Symbolic and Numeric Algorithms for Scientific Computing (SYNASC)*, 2020, pp. 17–23. doi: 10.1109/SYNASC51798.2020.00015.
- [8] L. Faubel *et al.*, 'A MLOps Architecture for XAI in Industrial Applications', in *2024 IEEE 29th International Conference on Emerging Technologies and Factory Automation (ETFA)*, 2024, pp. 1–4. doi: 10.1109/ETFA61755.2024.10711084.
- [9] N. Sirisha, A. Kiran, M. Arshad, and M. M., 'Automating ML Models Using MLOps', in *2024 International Conference on Advancements in Smart, Secure and Intelligent Computing (ASSIC)*, 2024, pp. 1–5. doi: 10.1109/ASSIC60049.2024.10507923.
- [10] S. Mäkinen, H. Skogström, E. Laaksonen, and T. Mikkonen, 'Who Needs MLOps: What Data Scientists Seek to Accomplish and How Can MLOps Help?', in *2021 IEEE/ACM 1st Workshop on AI Engineering - Software Engineering for AI (WAIN)*, 2021, pp. 109–112. doi: 10.1109/WAIN52551.2021.00024.
- [11] C. M. Yilmaz, 'MLOps in Practice: Applications, Challenges, and Insights Across Domains', in *2025 5th International Conference on Artificial Intelligence and Signal Processing (AISP)*, 2025, pp. 1–5. doi: 10.1109/AISP68263.2025.11396140.
- [12] I. Kumara, F. Pecorelli, G. Catolino, R. Kazman, D. A. Tamburri, and W.-J. Van Den Heuvel, 'Architecting MLOps in the Cloud: From Theory to Practice', in *2023 IEEE 20th International Conference on Software Architecture Companion (ICSA-C)*, 2023, pp. 333–335. doi: 10.1109/ICSA-C57050.2023.00076.
- [13] D. Panchal, P. Verma, I. Baran, T. Hsiung, D. Musgrove, and D. Lu, 'Reusable MLOps: Reusable Deployment, Reusable Infrastructure and Hot-Swappable AI models and services', in *2024 10th International Conference on Smart Computing and Communication (ICSCC)*, 2024, pp. 645–651. doi: 10.1109/ICSCC62041.2024.10690392.
- [14] I. Poloskei, 'MLOps approach in the cloud-native data pipeline design', *Acta Technica Jaurinensis*, vol. 15, pp. 1–6, Apr. 2021, doi: 10.14513/actatechjaur.00581.
- [15] B. M. A. Matsui and D. H. Goya, 'MLOps: Five Steps to Guide its Effective Implementation', in *2022 IEEE/ACM 1st International Conference on AI Engineering – Software Engineering for AI (CAIN)*, 2022, pp. 33–34. doi: 10.1145/3522664.3528611.
- [16] S. Garg, P. Pundir, G. Rathee, P. K. Gupta, S. Garg, and S. Ahlawat, 'On Continuous Integration / Continuous Delivery for Automated Deployment of Machine Learning Models using MLOps', in *2021 IEEE Fourth International Conference on Artificial Intelligence and Knowledge Engineering (AIKE)*, 2021, pp. 25–28. doi: 10.1109/AIKE52691.2021.00010.
- [17] L. K. Chikkala, N. K. Pathi, and S. Abhi, 'Automating MLOps Pipelines with GitHub Actions and Azure: A Case Study on Diabetes Prediction', in *2025 International Conference on Emerging Technologies in Computing and Communication (ETCC)*, 2025, pp. 1–6. doi: 10.1109/ETCC65847.2025.11108664.
- [18] D. Sundaram, 'AI-Driven Predictive Patient Monitoring System Using Kubeflow and CI/CD Enabled MLOps Framework', in *2025 International Conference on Computer and Applications (ICCA)*, 2025, pp. 1–6. doi: 10.1109/ICCA66035.2025.11430793.
- [19] R. B. Bahaweres, A. Zulfikar, I. Hermadi, A. I. Suroso, and Y. Arkeman, 'Docker and Kubernetes Pipeline for DevOps Software Defect Prediction with MLOps Approach', in *2022 2nd International Seminar on Machine Learning, Optimization, and Data Science (ISMODE)*, 2022, pp. 248–253. doi: 10.1109/ISMODE56940.2022.10180973.

- [20] M. Heydari and Z. Rezvani, ‘Challenges and Experiences of Iranian Developers with MLOps at Enterprise’, in *2023 7th Iranian Conference on Advances in Enterprise Architecture (ICA EA)*, 2023, pp. 26–32. doi: 10.1109/ICA EA60387.2023.10414442.
- [21] A. Bodor, M. Hnida, and D. Najima, ‘From Development to Deployment: An Approach to MLOps Monitoring for Machine Learning Model Operationalization’, in *2023 14th International Conference on Intelligent Systems: Theories and Applications (SITA)*, 2023, pp. 1–7. doi: 10.1109/SITA60746.2023.10373733.
- [22] H. Bhatt, S. Arun, A. Kakran, and K. Vaidhyathan, ‘Towards Architecting Sustainable MLOps: A Self-Adaptation Approach’, in *2024 IEEE 21st International Conference on Software Architecture Companion (ICSA-C)*, 2024, pp. 179–182. doi: 10.1109/ICSA-C63560.2024.00038.
- [23] Y. Billeter *et al.*, ‘MLOps as Enabler of Trustworthy AI’, in *2024 11th IEEE Swiss Conference on Data Science (SDS)*, 2024, pp. 37–40. doi: 10.1109/SDS60720.2024.00013.
- [24] N. Lang, ‘Ridge Regression: A Robust Path to Reliable Predictions’, Towards Data Science. Accessed: Jun. 05, 2026. [Online]. Available: <https://towardsdatascience.com/ridge-regression-a-robust-path-to-reliable-predictions-851290bed579/>
- [25] ‘What is lasso regression? | IBM’. Accessed: Jun. 05, 2026. [Online]. Available: <https://www.ibm.com/think/topics/lasso-regression>
- [26] Z. Zhang, ‘Introduction to machine learning: k-nearest neighbors’, *Ann Transl Med*, vol. 4, no. 11, p. 218, Jun. 2016, doi: 10.21037/atm.2016.03.37.
- [27] E. Kavlakoglu, ‘What Is Random Forest? | IBM’. Accessed: Jun. 05, 2026. [Online]. Available: <https://www.ibm.com/think/topics/random-forest>
- [28] ‘Introduction to Boosted Trees — xgboost 3.2.1 documentation’. Accessed: Jun. 05, 2026. [Online]. Available: https://xgboost.readthedocs.io/en/release_3.2.0/tutorials/model.html
- [29] ‘GridSearchCV’, scikit-learn. Accessed: Jun. 05, 2026. [Online]. Available: https://scikit-learn/stable/modules/generated/sklearn.model_selection.GridSearchCV.html
- [30] ‘VotingRegressor’, scikit-learn. Accessed: Jun. 05, 2026. [Online]. Available: <https://scikit-learn/stable/modules/generated/sklearn.ensemble.VotingRegressor.html>